

The model is the engine. The **harness** is the car.

What makes a good agentic coding harness, and why it matters more than the model.



SECTION 01 / 05

The Reality

Two years into the agent era, the honest read: the 2025 hype didn't land, and the model itself is commoditizing.

01

2025 was the “Year of the Agent.” It didn't land.

The hype was model-led. In production the model alone fell short, expensively, and the studies agree on the reason: it wasn't the model.

95%

of enterprise GenAI pilots showed **no measurable P&L impact**

MIT NANDA, 300+ deployments: the divide “is not driven by model quality” but by approach. The dividing line is memory, workflow integration, and learning.

80%

of AI projects fail, **twice** the rate of non-AI software

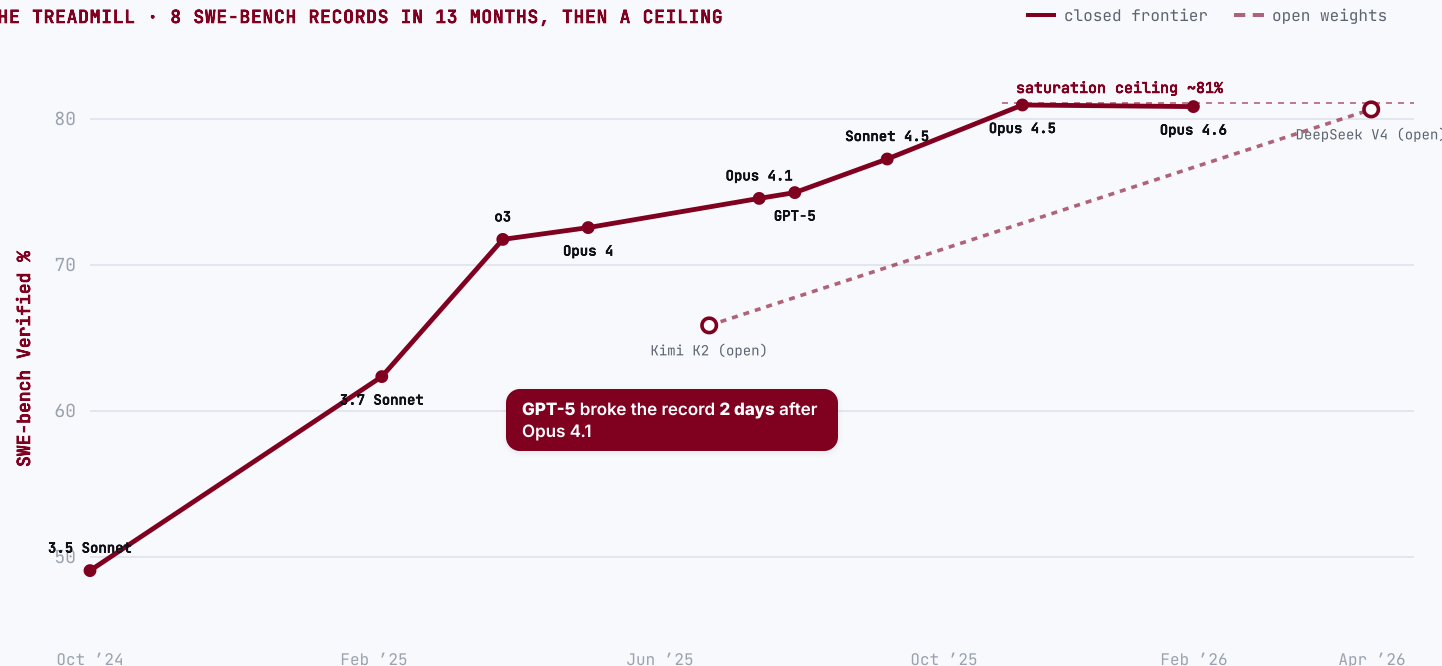
RAND, from 65 practitioner interviews: of five root causes, only one touches model capability. The rest are problem-alignment, data, and integration.

Across MIT and RAND, one verdict: the missing variable wasn't intelligence, it was the **harness around the model**.

Models leapfrog every benchmark, then commoditize.

A new frontier model tops every benchmark... until the next does, **days** later. Then it saturates ~81%, open weights reach **parity**, and the benchmark itself gets retired.

THE TREADMILL • 8 SWE-BENCH RECORDS IN 13 MONTHS, THEN A CEILING



🔗 Open weights, now at parity

≈ 80.6% = the frontier

DeepSeek V4-Pro matches the closed SOTA on SWE-bench Verified (Apr '26). Composite gap now ~4 months / 8 ECI pts (Epoch AI, May '26).

⚠️ The benchmark itself broke

59% of hard tests flawed

OpenAI retired SWE-bench Verified (Feb '26): frontier models had trained on the answers; it's saturated. → SWE-bench Pro.

📉 Inference is collapsing

≈ 280× cheaper / 18 mo

GPT-3.5-level tokens: \$20.00 → \$0.07 per M. ~10× every year: "LLMflation."

"The models are getting commoditized."

Satya Nadella • Microsoft CEO • B2 Pod, Dec 2024



The model is a commodity, not a moat.

Whatever tops the leaderboard this week is matched, undercut and open-weighted within months, and the benchmark itself gets retired.

SECTION 02 / 05

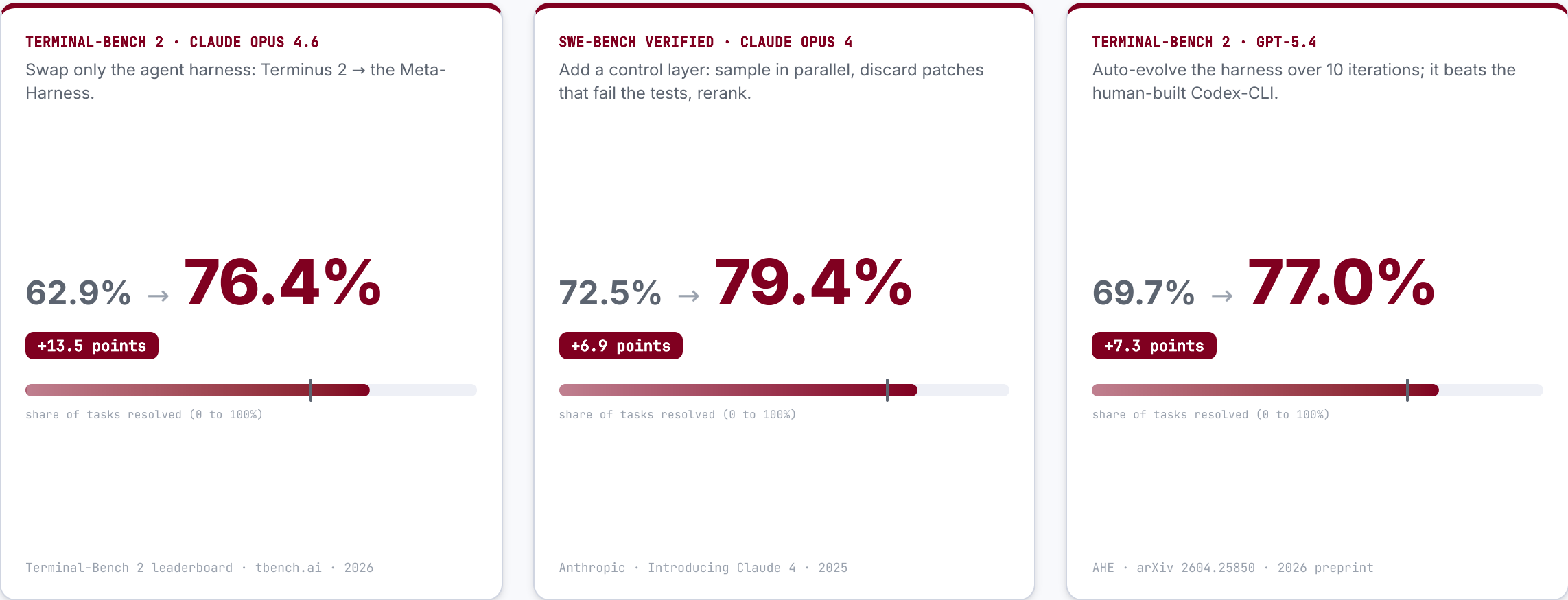
The Agentic **Harness**

The pilots that stalled tuned the model. The ones that shipped engineered the harness.

02

Hold the model fixed, change the harness.

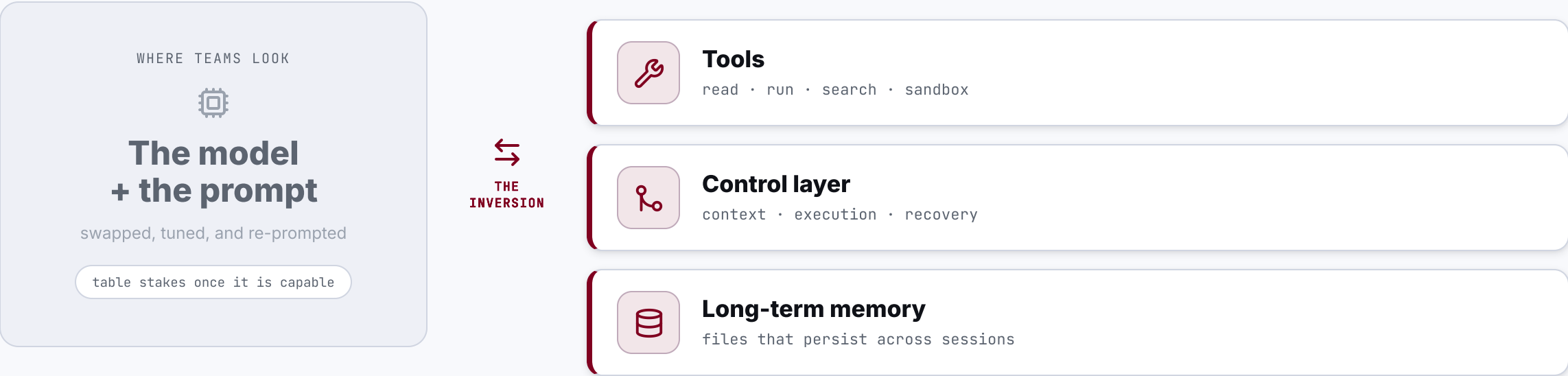
Three current frontier models, 2025 to 2026. In each, the model weights never change; only the harness around them does.



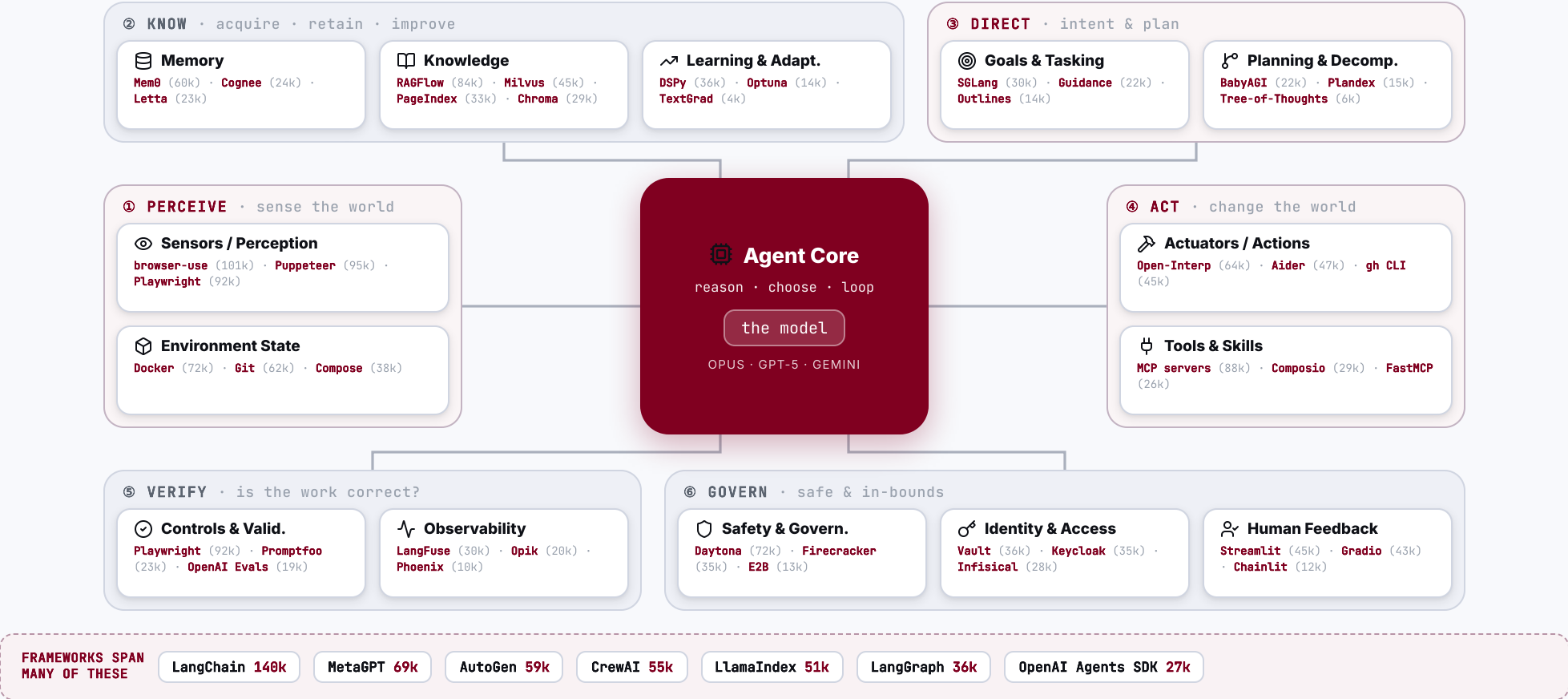
Same weights every time. The harness is the differentiator, **not the model**.

Performance lives in the tooling around the model, not the model.

The effort runs one way; the leverage runs the other. Teams tune the prompt, but performance comes from the tooling, control, and memory wrapped around the model.



The agent core, wrapped in 14 capabilities, and who leads each.



Do you really think software engineering is dead?

That map was **50 tools** wrapping a single model: each its own system, all to be wired together and kept alive.

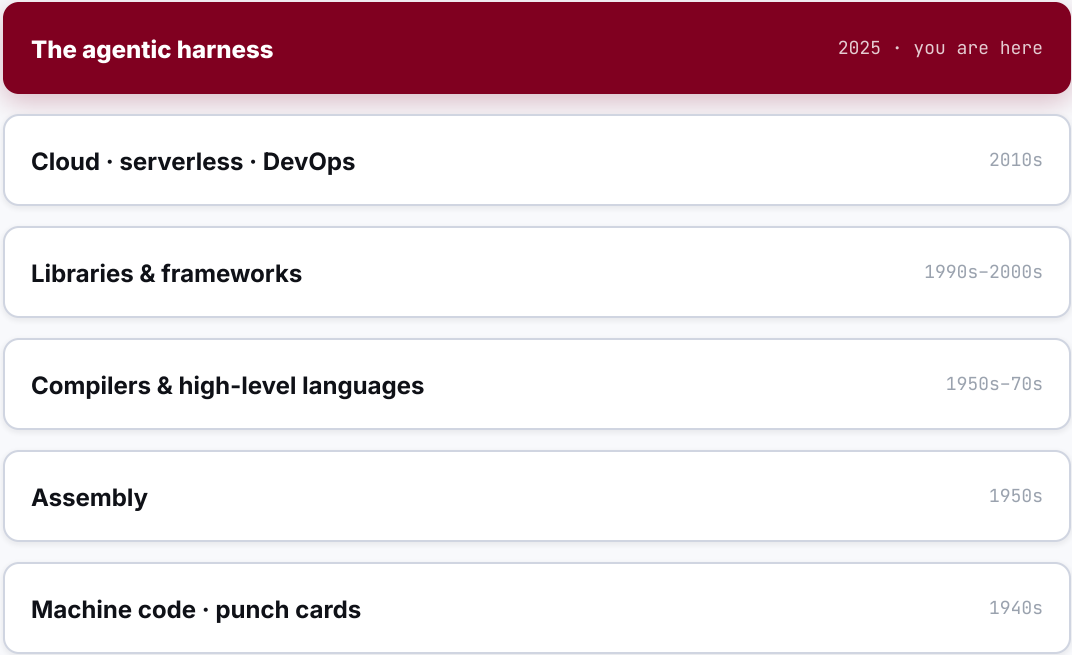
50 tools around one model
and only the top few per capability; 80+ ship in the wild

 **Each tool is its own system**
its own API, config, failure modes and learning curve

 **Orchestration is the hard part**
wiring 50 moving parts into one reliable, debuggable loop

 **Maintenance never stops**
versions drift, tools break, the harness needs constant upkeep

THE LADDER OF ABSTRACTION · every rung was once “the end of programming”



 **No. We’ve just climbed another rung on the ladder of abstraction.**
Software engineering didn’t disappear; building and maintaining this harness is the engineering now.

HOW THE AGENT REACHES THE WORLD

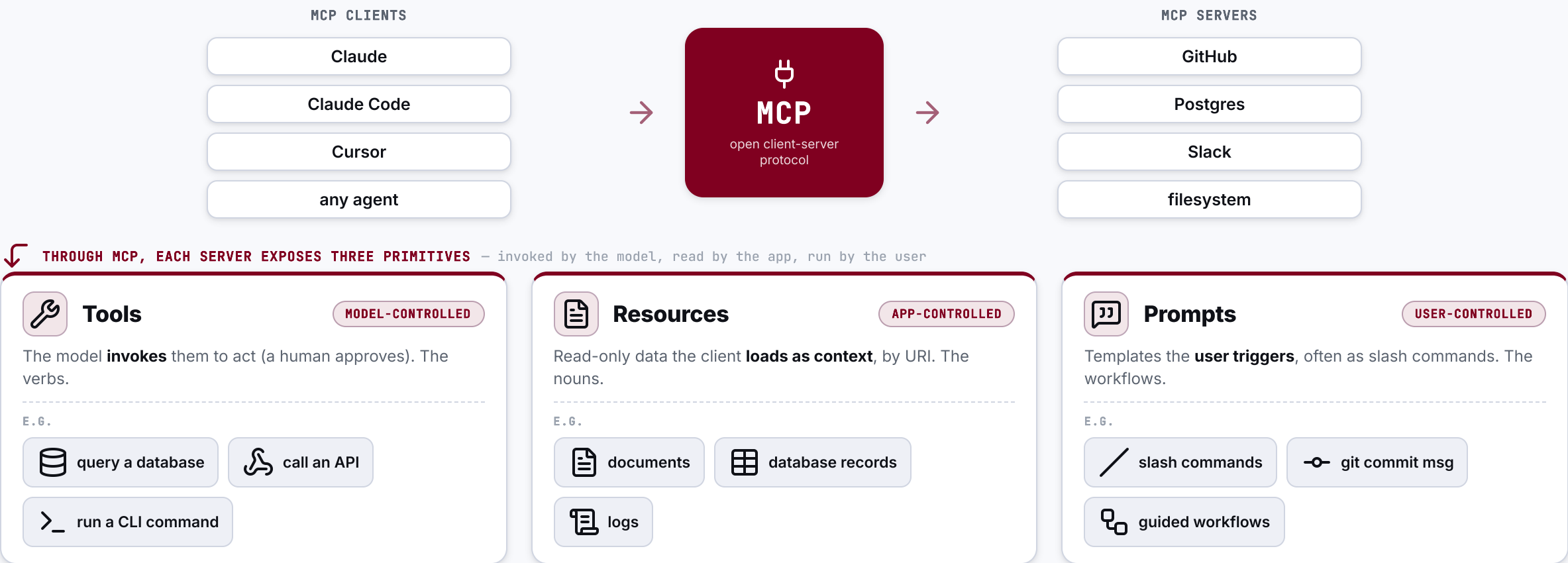
Tools

An agent is only as capable as the tools it can call. There are three ways to hand it some: the CLI, MCP, and a direct API. They are not rivals to pick between; each pays off in a different place, and the harness wires up all three.



MCP: one protocol for every tool an agent can reach.

The open standard Anthropic shipped in November 2024. Think USB-C for AI: expose a tool through one server, and every MCP client can call it.



 **Solves the N-by-M problem.** Without a standard, M apps each wire N tools → M×N bespoke connectors. With MCP it is M + N: write a server once, every client uses it.

Most of MCP is a wrapper that makes CLI & API tools *discoverable*.

Peel back a typical MCP server and you find a shell command or an HTTP call it already had. What MCP adds on top isn't capability, it's a self-describing, standardized way for the agent to **find** and call it.

PEEL BACK AN MCP SERVER

 MCP protocol · the discoverability skin

JSON-RPC · `tools/list` · JSON schemas

 WHAT ACTUALLY RUNS

<code>create_issue</code>	→ <code>gh issue create</code>	CLI
<code>query</code>	→ <code>POST /v1/sql</code>	API
<code>search_files</code>	→ <code>rg <pattern></code>	CLI

 A thin translation shim: same command, wrapped in a schema.

 What it adds — discoverability

-  **Self-description.** `tools/list` hands back names + typed schemas, so the agent calls a tool it has never seen.
-  **Standardization.** One protocol, reusable across every client, plus auth and a common error shape.

 What it doesn't — the capability

-  The CLI or API still does the work. A bare `gh` or `curl` does the same thing, with no schema to load.

 So the question is rarely “CLI or MCP?” but “does the agent need to **discover** this tool at runtime?” If it already knows the command, the CLI is enough; MCP earns its context cost when cross-client discoverability does.

THE HARDEST LAYER

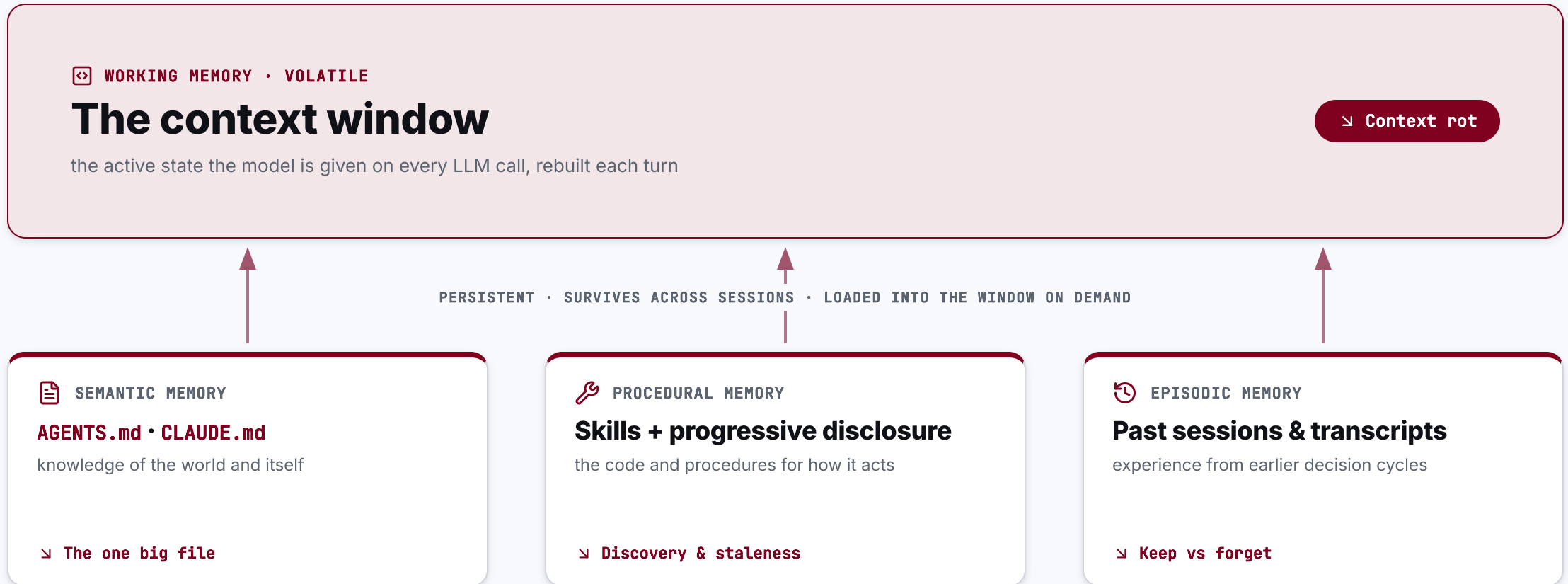
Memory

Bolt-on memory is now commodity: Mem0, Letta, vector search. What no tool ships is memory that holds its structure and traceability across refactors, and that is what the harness has to build.



An agent has four memories. One is volatile, three persist.

The **CoALA** framework (Princeton, TMLR 2024) names them. Everything the agent knows is loaded through the one volatile window to be used.

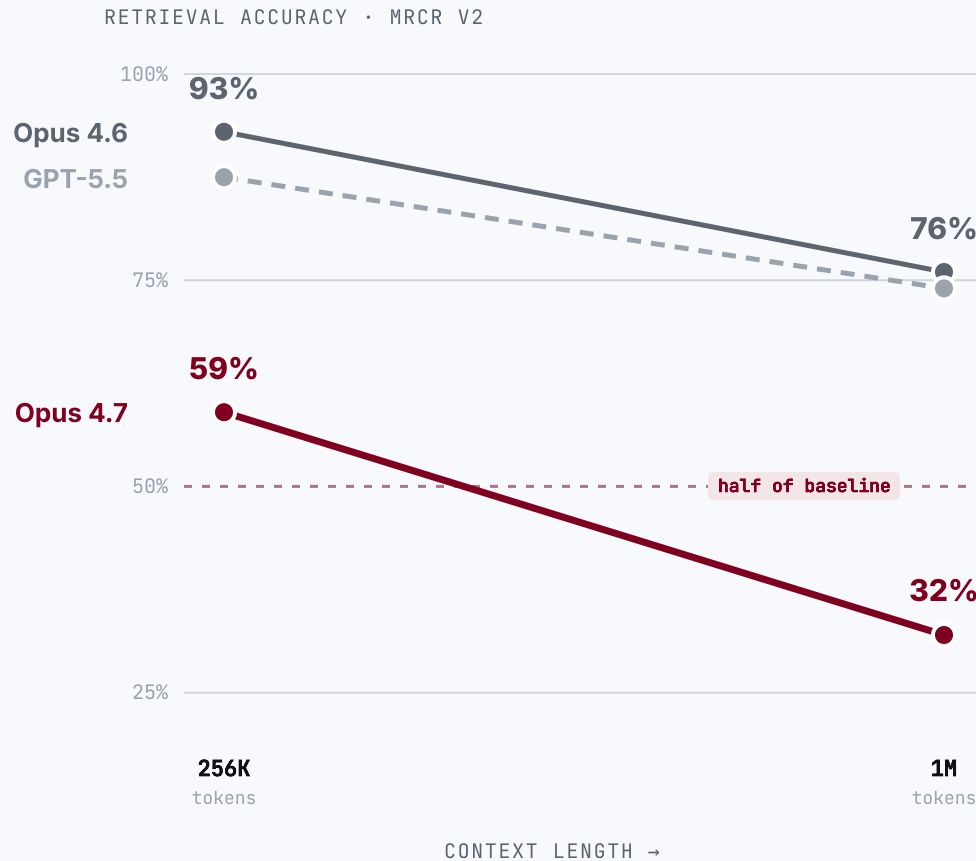


One volatile window, three persistent stores. Each fails in its own way, next four slides.

No. A bigger context window is not bigger memory.

Working memory is the live context window: the only state the model actually sees, and it degrades as you fill it.

WHERE IT BREAKS



The advertised window is **not** the usable window.

59% → 32%

Claude Opus 4.7 loses ~half its multi-needle retrieval from 256K to 1M tokens. Opus 4.6 holds better (93% → 76%).

Codex too

the GPT-5.x line behind Codex CLI drops ~87% → 74% at long context, a proxy; no per-length figure is published for the Codex-tuned model.

11 of 13

models fell below half their accuracy by 32K tokens. Context rot isn't new (NoLiMa, 2025).

→ **The fix:** curate, don't cram. Retrieve only the few high-signal tokens the task needs into a fresh window; treat context as a budget, not a bucket.

Writing it down isn't memory either.

Semantic memory is what the agent knows about your project: the `AGENTS.md` and `CLAUDE.md` it reads in at the start of every session.

WHERE IT BREAKS

OpenAI built a 1M-line product with zero hand-written code. At that scale, they found one big `AGENTS.md` broke down, four ways:

-  **Crowds out the task**
-  **Too much guidance, none**
-  **Rots silently**
-  **Can't be verified**

 **AGENTS.md**

ONE BIG FILE

Project setup

Build & test

Code style

+ 900 more lines • no one can tell which still apply

→ **OpenAI's fix:** they replaced it with a ~100-line table of contents over a structured `docs/` tree. Memory has to be a **checkable map**, not a flat file you hope holds.

Skills tame context bloat, but only if the agent picks the right one.

Procedural memory is the agent's reusable know-how: skills, surfaced by progressive disclosure and loaded just-in-time on a matching prompt.

HOW A SKILL LOADS

AGENT CONTEXT · SYSTEM PROMPT

skill-a

skill-b

skill-c

+ 97

names + descriptions only, always loaded (cheap)

↪ user prompt matches **skill-b** → load it

SKILL-B, JUST-IN-TIME

SKILL.md full instructions

ON MATCH

scripts / references

ON DEMAND

WHERE IT BREAKS

- ✖

Discovery rides on one sentence. Claude picks from 100+ skills using only each description; a vague one never fires.
- 🛒

The metadata is never free. Every description permanently shares the context budget.
- 🌀

It rots silently. Bundled instructions drift; the only guardrail is "avoid time-sensitive info."
- 📦

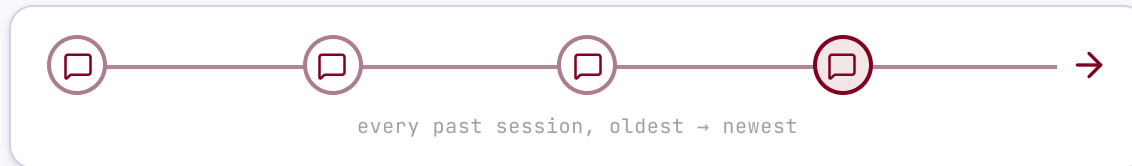
The eager alternative is worse. Codex **AGENTS.md** and Aider load every turn, no relevance gate.

→ **The fix:** a curated, sharply-described, version-controlled skill set. The procedure layer rewards **structure**, not accumulation.

The hard part isn't remembering. It's forgetting.

Episodic memory is experience from past sessions: the transcripts of what the agent did, and what they taught.

WHAT IT CAPTURES



✧ DISTILLED INTO

📅 KEPT AS EPISODIC MEMORY

💬 Past interactions

Conversation keywords

✅ What worked

❌ What didn't work

WHERE IT BREAKS

🕒 **Keeping it all is too costly.** >90% of tokens go to re-stuffing full history each turn.

🔍 **More memory, noisier recall.** The more you retain, the harder the right memory is to find.

📉 **Stale memories mislead.** Old context isn't just bloat; it points the agent the wrong way.

❓ **What to forget is unsolved.** Even scored by importance, recency and relevance, "learned forgetting" is still open (2026).

→ **The fix:** treat retention as a decision, not a default. Choosing what's worth keeping needs a **model of what matters**, which is exactly what no off-the-shelf store has.

SECTION 03 / 05

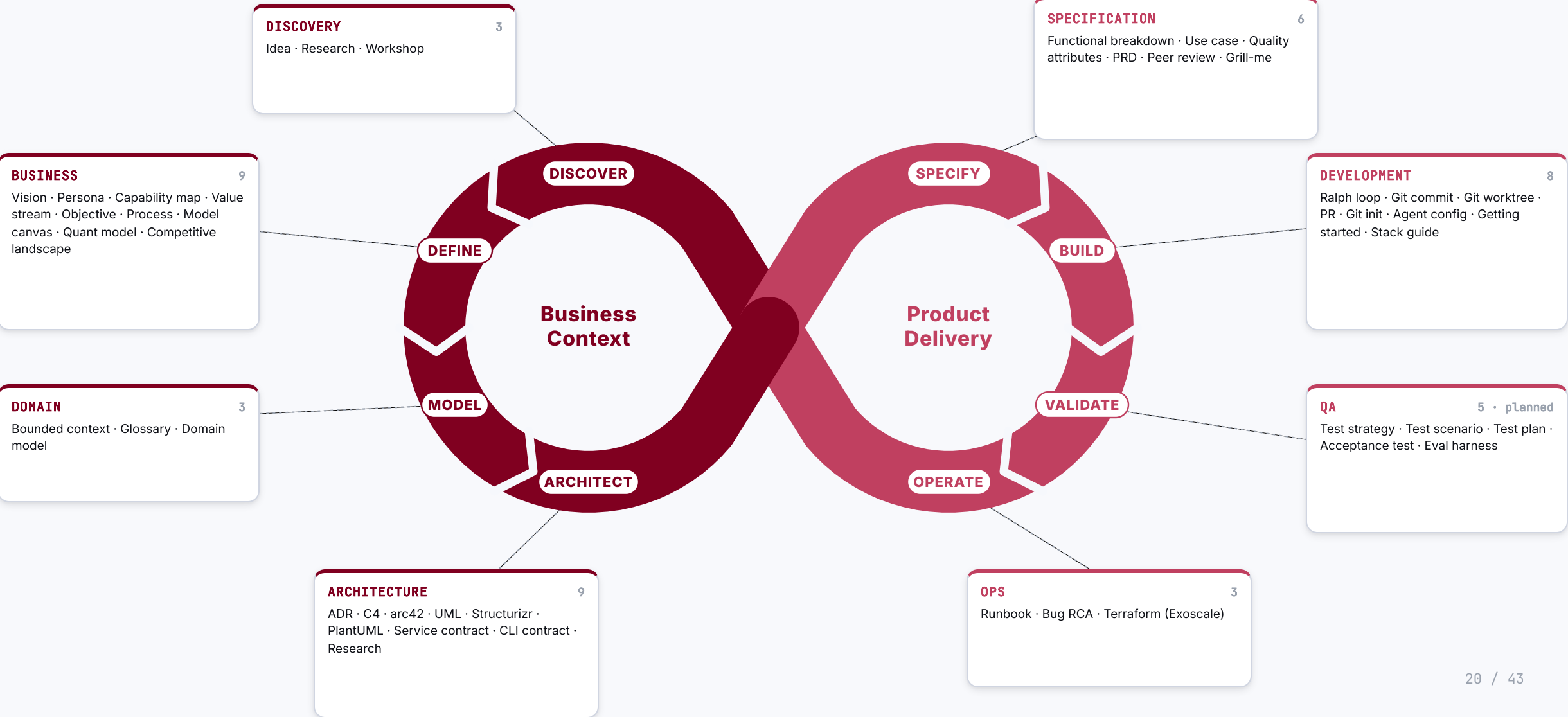
The Metamodel

How an agent armed with skills and rules turns business context into deployable artefacts, every step traceable.

03

Skills & rules wrap the whole lifecycle, from discovery to operations.

Far beyond DevOps: eight phases from business discovery to operations, each owned by a prefix, all tracing back to one metamodel.



One specimen file, *dissected*: OKF below, skill discipline on top.

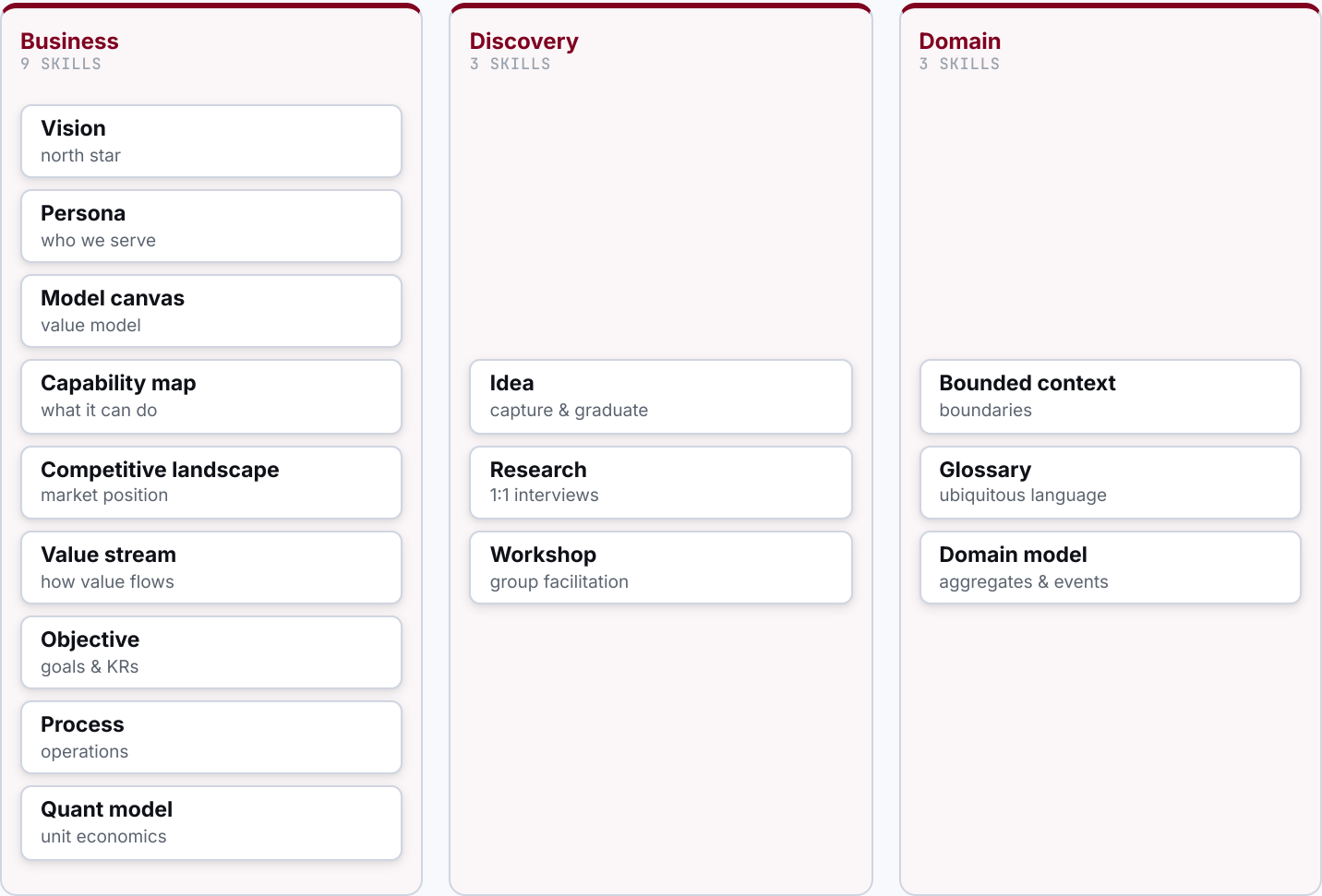


OKF makes it portable; the skill makes it **true**.

Discover & Define • make the context real.

WHAT THE AGENTIC ENGINEER DOES HERE

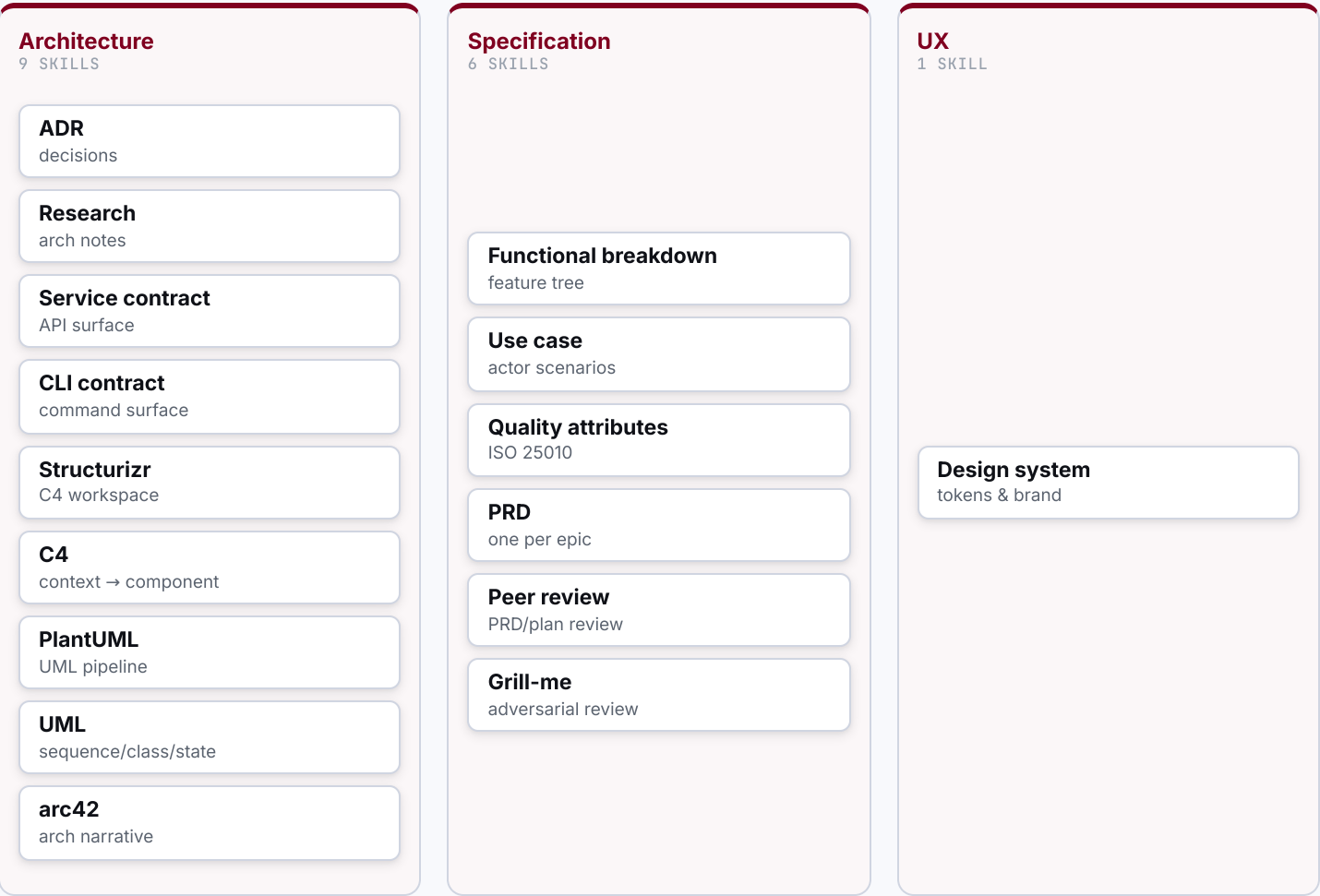
- **Brainstorm & capture**
ideas, hypotheses
- **Research & scan**
market, competitors
- **Interview & workshop**
with the business
- **Model the business**
vision → objectives
- **Define the domain**
ubiquitous language



Architect the solution · turn context into a solution.

WHAT THE AGENTIC ENGINEER DOES HERE

- **Decide architecture**
ADRs, C4, arc42
- **Define contracts**
service, CLI
- **Break down function**
FBS
- **Specify & qualify**
use cases, quality
- **Write & review PRDs**
one per epic



Execute & Deliver • plan, build, test, ship.

WHAT THE AGENTIC ENGINEER DOES HERE



Sequence the roadmap
epics, phases



Plan increments
atomic steps



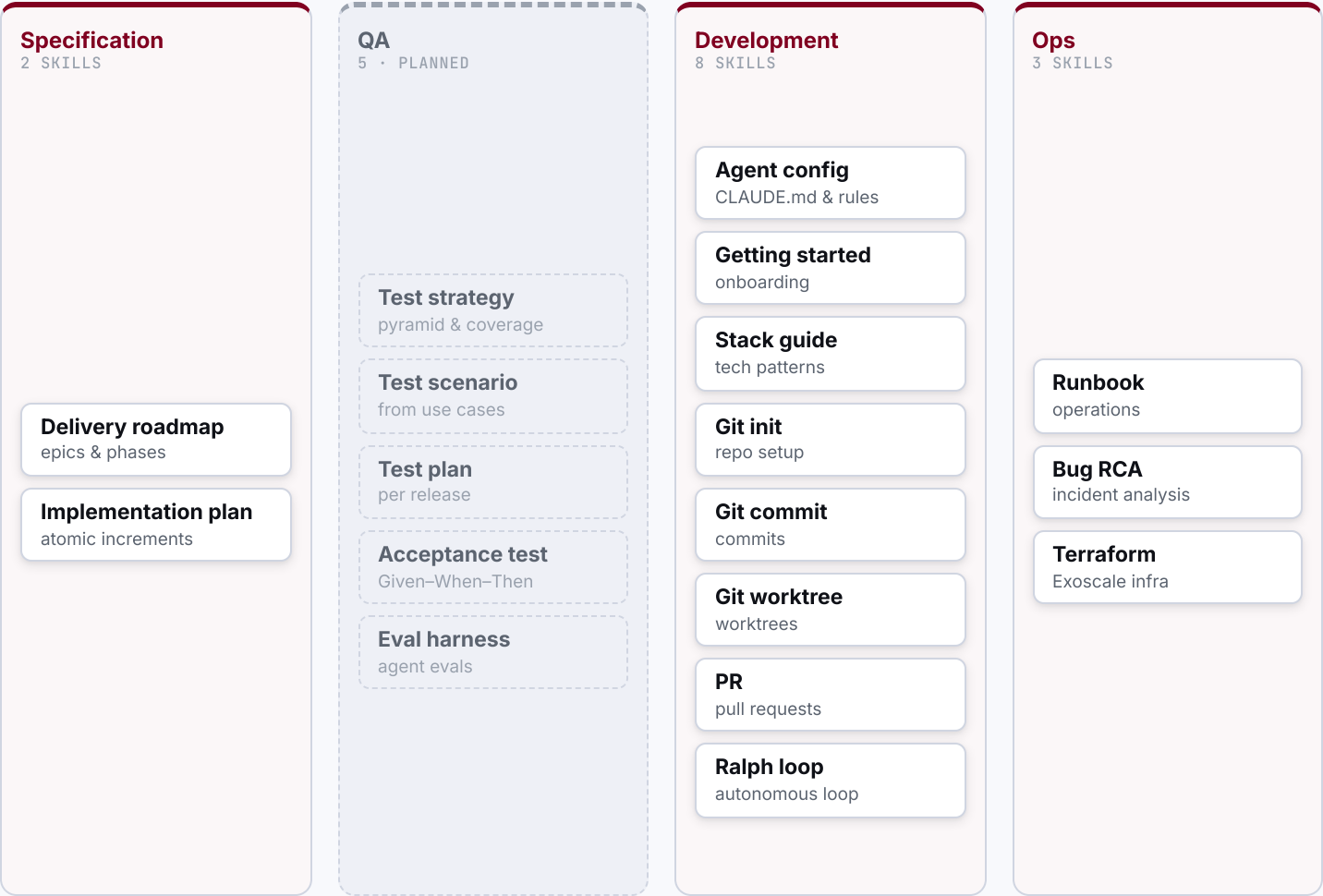
Build & validate
tests, checks



Deploy
infra, release



Operate & fix
runbooks, RCA



SECTION 04 / 05

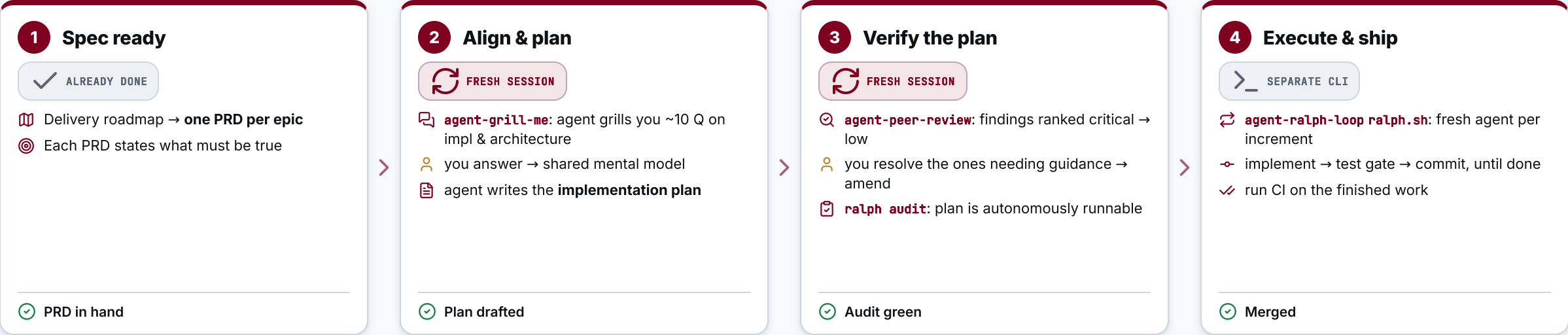
The Build **Workflow**

From a PRD to merged code: how one engineer drives agents through alignment and verification gates, then lets them run autonomously.

04

From PRD to merged code: two fresh-session gates, then autonomy.

Business context and discovery are done; the roadmap gives one PRD per epic. The loop below runs per PRD.



Two **fresh-session** checks (grill-me before, peer-review after) keep me and the agent aligned; the loop only runs once the plan is proven autonomous.

PRD says what must be true; the plan says how, safely.

Two artefacts, one chain: the PRD scopes the epic, the implementation plan breaks it into runnable increments.

BUILD BY FEATURE · ONE PER EPIC

PRD

"What must be true, and why."

-  **Problem, goals & non-goals:** the scope boundary
-  **User stories** grounded in personas (P-NN)
-  **Acceptance criteria** + success metrics
-  **\$0 traceability:** the epic and its dependencies

E-NN · P-NN · C-N.M.FXX · QA-XXNN · UC-NN



1 PRD → 1 plan

acceptance criteria become the
plan's test gates

increments deliver the PRD
scope

ATOMIC INCREMENTS · ONE PER PRD

Implementation plan

"How we get there, reversibly."

-  **Ordered increments** (Inc-1 ... Inc-N)
-  each **small** · **testable** · **reversible**
-  scope + primary files per increment
-  **test gate** + exit criteria = done

Plan-NNNN · Inc-N · Status: pending → done

Two fresh agents pressure-test the work, before and after the plan.

Grill-me pulls the thinking out of my head; peer-review checks the document. Both run in a clean session.



FRESH AGENT

The PRD says the importer is “fast.” What’s the p95 budget, and on which dataset?

✓✓

p95 under 200 ms on the 10k-artefact repo, asserted in CI.

✓✓



YOU

ALIGNMENT · BEFORE THE PLAN

agent-grill-me

Live Socratic session on the PRD, one question at a time.

- Agent asks ~10 focused Q on **implementation & architecture**
- Vocabulary checked against the **domain glossary**
- Decisions crystallised into **ADRs** while context is live
- Surfaces the **thinking gaps** only questioning reveals

Output: **shared mental model** → the agent then writes the implementation plan.

FINDINGS, RANKED CRITICAL → LOW

- critical** Inc-3 ships no rollback path
- major** No test gate for the p95 latency budget
- normal** “trace” used before the glossary defines it
- low** Inc-4 and Inc-5 could run in parallel

VERIFICATION · AFTER THE PLAN

agent-peer-review

Static, independent scan of the finished plan.

- Reads the plan end-to-end, builds a requirement map
- Every finding carries **concrete remediation**
- I resolve the ones needing guidance → **amend the plan**

Output: **validated plan** → ready for the autonomy audit.

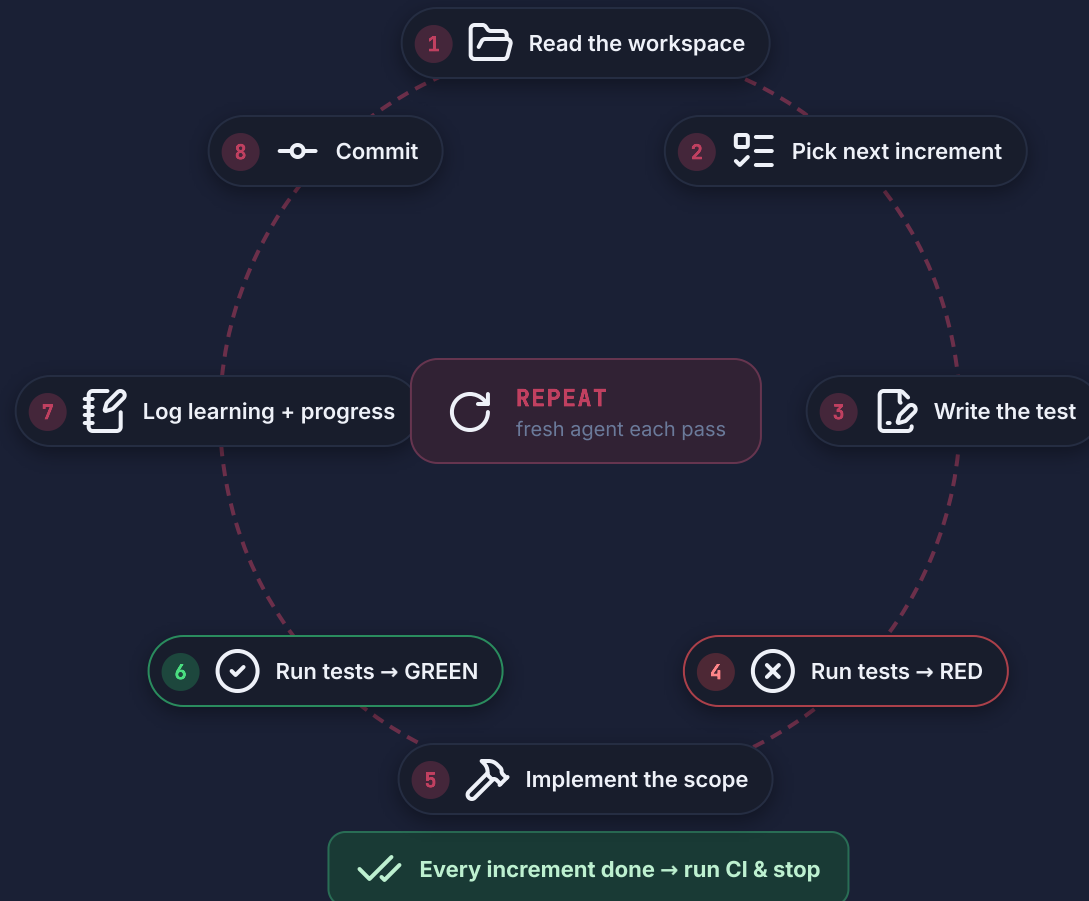
The Ralph loop: a fresh agent per increment, until the plan is done.

WHAT THE LOOP IS

- ↻ A bash script (**ralph.sh**) drives the plan to completion, one increment per iteration.
- ↻ **Fresh agent each iteration** → clean context, no drift or context rot.
- 🚩 Loops while increments are **pending**; stops when every one is **Status: done**.

```
ralph.sh <workspace> \  
  --max-iterations 50 --with-prd --with-push  
# spawns: claude --print < iteration-prompt
```

RALPH LOOP • ONE INCREMENT PER PASS



BUILDING WITH AGENTS • IN PRACTICE

Dev Tooling

The editor, model, agent-CLI and IDE you build with, open-source or proprietary, and how engineers climb from vibe-coding to real agentic engineering. The harness and the method underneath stay the same.



Open-source or proprietary: **your setup, your call.**

Three tiers: bring a model, compose a custom harness, or run a full IDE, each available **fully open** or **fully proprietary**. What doesn't change is the method underneath.

	 OPEN-SOURCE · self-hostable	 PROPRIETARY · hosted
 Full IDE ALL-IN-ONE	VS Code +187k Zed +86k Continue +35k Tabby +34k Zed standalone · VS Code + ext	Cursor Windsurf Antigravity Kiro closed-source · bundled
 Custom harness EDITOR + AGENT	opencode +180k Cline +64k Goose +50k Aider +47k Crush +26k open + model-agnostic	Claude Code +135k Codex +94k Copilot Antigravity CLI Amp proprietary · vendor-hosted
 Inference & routing OPTIONAL	Ollama +175k llama.cpp +118k vLLM +85k LiteLLM +52k self-host · open	OpenRouter Groq Hugging Face Bedrock Vertex AI Azure OpenAI managed · cloud gateway
 Bring your own model THE MODEL	DeepSeek Llama Qwen gpt-oss Mistral open weights · self-hostable	Claude GPT-5 Gemini 3 Grok frontier · hosted

 **The method**
THE CONSTANT, EITHER SIDE

spec

 →

plan

 →

grill-me

 →

peer-review

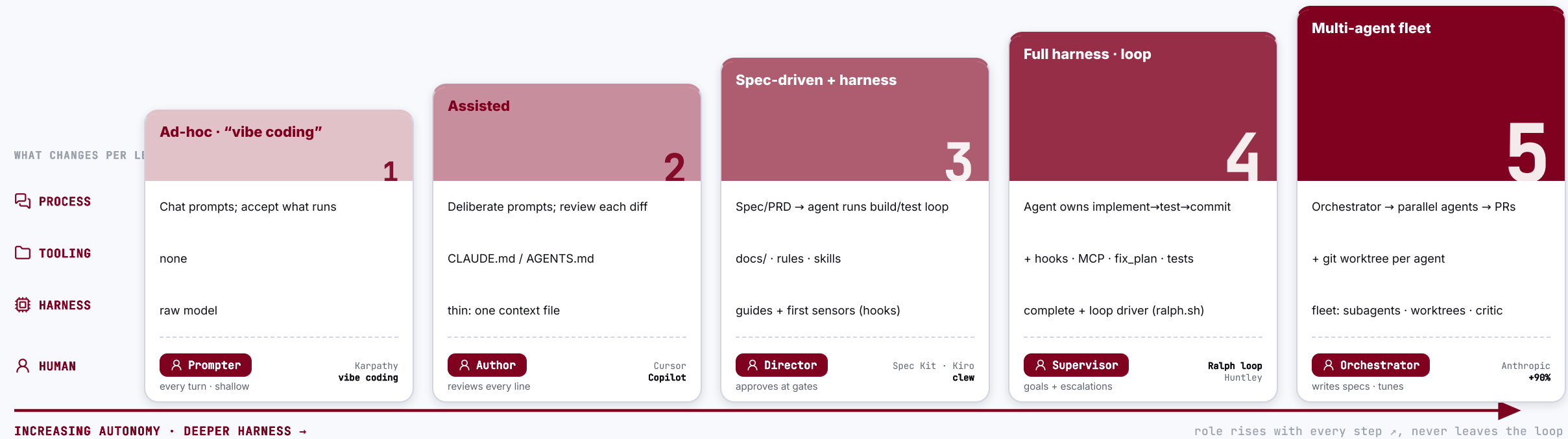
 →

loop

 · artefacts · rules · skills · the metamodel

From “vibe coding” to *agentic engineering*: five levels.

Autonomy climbs and the harness thickens, but the human moves **up** the stack (prompter → orchestrator), never out of it.



⚠ **Even L5 is *supervised* autonomy, not hands-off.** METR: experienced devs were 19% *slower* on mature codebases; SWE-bench Pro tops out ~23%; Ralph’s author: “no way I’d use it on an existing codebase.” The guardrail against Thoughtworks’ “**cognitive debt**” is keeping humans on standards, architecture & final sign-off.

BEFORE YOU GO

Key takeaways

Ten commandments for the developer, ten for the company. What to actually do on Monday.



Ten commandments for the **developer**.

The agent is a power tool. The judgement, the architecture and the guardrails are still yours.

TOOLING & HARNESS

- 1 Build your own harness, but share what works.**
No one tells you which IDE or agent to pick; pass the patterns and best practices back to the team.
- 2 The tooling is hackable.**
It was built to be bent; adapt it to your workflow.
- 3 Start minimal; earn every tool and framework.**
Hit a wall before you add a tool; run the agent loop bare before you adopt an agentic framework.

WORKING WITH THE AGENT

- 4 Treat your agent like a smart junior.**
Give it tools, steer it, correct it.
- 5 Keep a fresh context.**
Start clean; don't let the window rot.

CRAFT & JUDGEMENT

- 6 Fast to write is not a reason to write.**
Coding was never the bottleneck; thinking was.
- 7 Trust, but verify.**
Read the diff; every line is yours to defend.
- 8 Learn by playing.**
Build hobby projects, not experiments on company goals.

ARCHITECTURE

- 9 Agents magnify bad architecture as fast as good.**
So build the guardrails.
- 10 Refactoring got cheap. Architecture didn't get optional.**
Cheap to change is not the same as fine to wing it.



Engineer the harness, not just the prompt. The craft moved **up a rung**; it didn't disappear.

Ten commandments for the **company**.

Building with agents is an organisational capability, not a gadget. Set the conditions.

ENABLEMENT & METHOD

- 1 Enable the tooling.**
Give every dev agents, sandboxes and access; don't gatekeep the lever.
- 2 Standardize the workflow and artefacts, not the tool.**
Mandate the method; let people pick their own harness.

GOVERNANCE & METRICS

- 3 Govern the data before the agent sees it.**
Decide what company data an agent may ingest; first, not after.
- 4 Measure what must be true, not velocity.**
Track traceability and correctness, not tickets closed.
- 5 Pay down cognitive debt.**
Don't let agents quietly erode the team's understanding.

REALITY & INVESTMENT

- 6 AI skips none of your real problems.**
Quality, process, planning, architecture are still there; you just hit the wall faster, and it's bigger.
- 7 Budget for maintenance.**
The harness is a living system, not a one-off.

STRATEGY & INDEPENDENCE

- 8 Claim AI sovereignty now.**
Don't turn every model and tool into a US dependency.
- 9 Default to open source.**
OSS always ends up the default; don't repeat the Microsoft Office lock-in.
- 10 The method is the moat, not the model.**
Invest in the harness and the memory; models are commodities.



The model is a commodity. Your edge is the **method and memory** you build around it.

THE TAKEAWAY

Engineer the **harness**, not just the prompt.

The model is the commodity. Your edge is the method, the memory and the guardrails you build around it.

Same model, better harness

The method is the moat

Standardize the method, swap the parts

Software engineering isn't dead. We just climbed **another rung** on the ladder of abstraction.

OPEN SOURCE • BUILT WITH AGENTS, OF COURSE

Found it useful? **Star the repos.**

The toolkit and the metamodel behind this talk are both public. Fork them, break them, and send the patterns that work back.

homemade-claude-kit

The skills & metamodel toolkit — the build order, artefacts and Claude skills that produced this deck.

☆ github.com/VictorHueni/homemade-claude-kit

clew

Product intelligence & architectural memory — the referentially-enforced metamodel itself, source of truth.

☆ github.com/VictorHueni/clew

REFERENCE & DEEPER CUTS

Appendix

For Q&A: how the tool mechanisms compare and combine, LSP as a capability, the memory-tooling landscape, and Karpathy's LLM Wiki pattern.



CLI, MCP & API: three ways in, *combined* per task.

Same goal, different sweet spot. The harness uses all three, matched to the job, then minds MCP's one real tax.

>_ CLI

shell + filesystem, via a Bash tool

🎯 The local build / test / commit loop

- ✓ Composable, token-cheap, tools the model already knows (git, docker)
- ✗ Must be installed; unstructured text; no discovery

🔌 MCP

a standardized server protocol

🎯 Standardized access to external systems

- ✓ Discoverable & reusable across clients; wire once, every app uses it
- ✗ Tool defs loaded upfront → context bloat

🔗 API

a direct HTTP / REST / SDK call

🎯 Hot paths where control & speed matter

- ✓ Full control, best latency, exact bespoke fit
- ✗ Bespoke wiring per service (the N×M tax); you own auth & retries



One agent · three mechanisms · chosen per task

⚠️ MCP's one real tax: context

50K–134K tokens of tool defs loaded before the request is read; task success falls **62→44%** when all 65 servers load at once. [arXiv 2508.16260](#)

⏴ The fix: progressive disclosure

Load tools **on demand** (code execution / Tool Search), not all upfront. **~85%** less context; tool-selection **49→74%**. [Anthropic](#)

LSP: a capability the agent reaches, not another transport.

The Language Server Protocol gives a model IDE-grade understanding of code. It is reached **through** CLI, MCP or API, and it is the protocol MCP's own spec says it took inspiration from.

WHAT LSP GIVES AN AGENT

user.py

```
1 class User:
2     def save(self) → None: ...
3     def has_role(self, r: str) → bool:
4     def email(self) → str: ...
5
6 u = User()
7 u.save()
8 u.has_role("admin")
9 print(u.email())
```

Go to Definition F12

Find All References F12

Peek Definition

Rename Symbol F2

 **grep / CLI** returns **text** matches, line by line.

 **LSP** returns the program's **semantic graph**: what is defined where, what calls what, what is type-broken.



Capability, not transport

RUNTIME · HOW YOU REACH IT

TRANSPORT

how

CLI

MCP

API

CAPABILITY

what

LSP

database

web search

files

LSP lives in the **what** layer. The agent reaches it through any transport above, e.g. an **lsp-mcp** server. The transport is plumbing; LSP is the ability.



What inspired MCP

DESIGN HERITAGE · NOT A CALL PATH

LSP M editors × N languages → **M + N**

MCP M agents × N tools → **M + N**

MCP's spec says it **"takes some inspiration from"** LSP: it shares the client-server JSON-RPC and the same M-by-N to M+N bet, refocused from code analysis to tool execution.

There's a tool for every memory. The recall still breaks.

Map the ecosystem onto CoALA's four memories and every layer is covered. Then the long-term, cross-session benchmarks turn ugly.

WHAT SHIPS TODAY, PER MEMORY TYPE

**Working memory**
LLMLingua · context editing · compaction · RAG

**Semantic memory**
Pinecone · Chroma · Mem0 · Zep / Graphiti · Cognee

**Procedural memory**
Agent Skills · Cursor Rules · MCP

**Episodic memory**
Letta · Zep · Mem0 · Windsurf memories

⚠ THE MEASURED GAP

~30%

accuracy that commercial assistants and long-context LLMs **lose** recalling across sustained sessions. Up to **60%** on the hard set.

LongMemEval (ICLR 2025) · LoCoMo

*"Current systems oscillate between **hoarding** (store everything, drown in noise) and **amnesia** (compress aggressively, lose rare but vital facts)."*

2026 memory survey, §9

NOT EVEN MEASURED

No benchmark tests memory that keeps its **structure and links valid across a refactor**. The field still "lacks a community-standard evaluation harness."

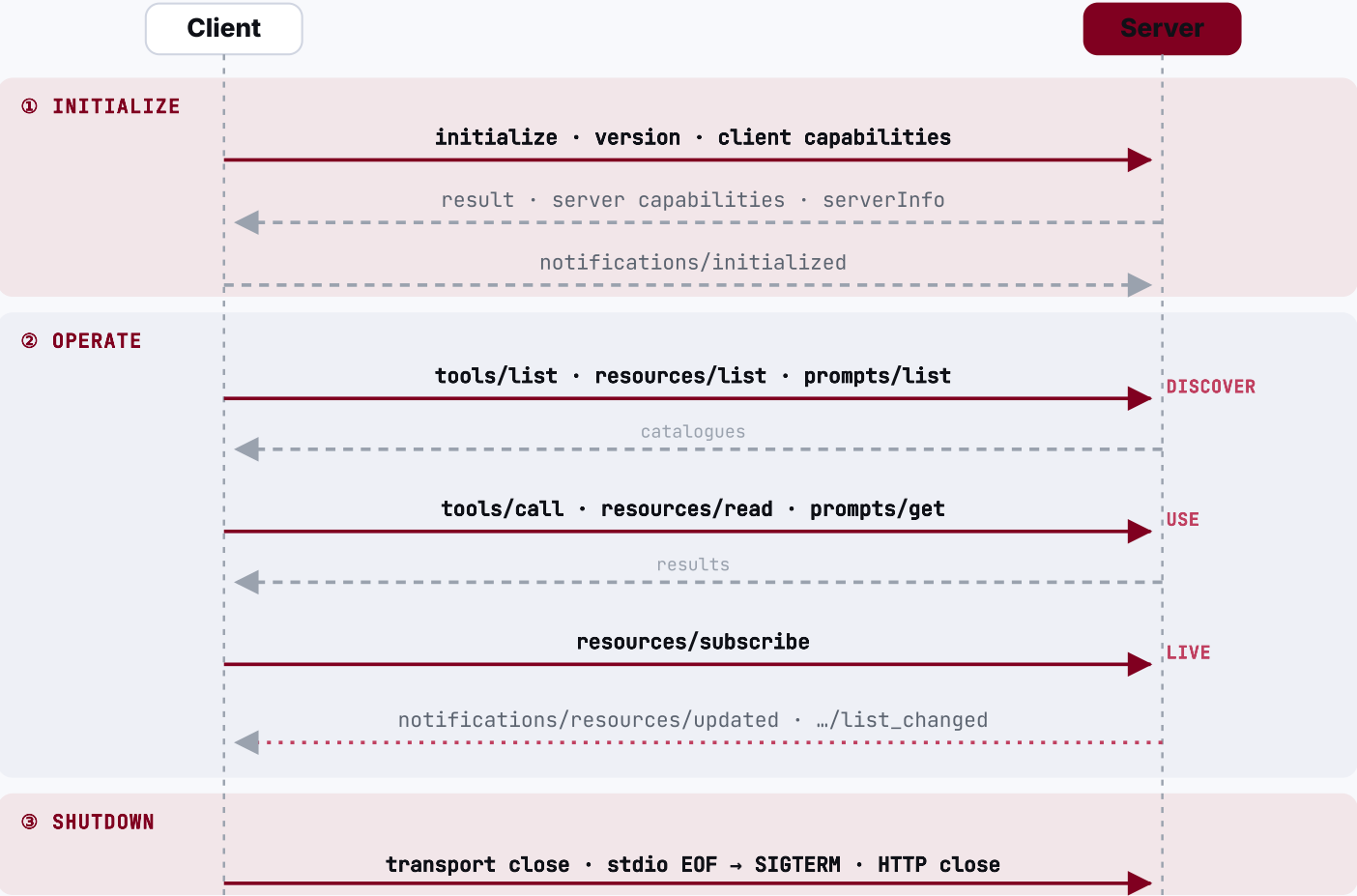
The LLM Wiki is a loop that *compounds*: every answer makes it smarter.



 **Compile once, not per query.** Ask a question and the wiki now knows the answer too, knowledge that accrues instead of being re-derived each turn.

One JSON-RPC session: *negotiate, operate, notify*.

Over stdio or HTTP. Capabilities are negotiated once at `initialize`; then it's discovery, use, and change notifications, until the transport closes.



 **Capability negotiation** · once, at initialize

Client declares

roots

sampling

elicitation

Server declares

tools

resources

prompts

logging

Sub-caps gate the extras: `listChanged` (tools · resources · prompts) and `subscribe` (resources only). Both sides use **only** what was negotiated.

 **Kept live by notifications** · server → client



`notifications/resources/updated` → the client re-reads that resource.



`.../list_changed` → the client re-lists tools, resources or prompts.

No polling: the session stays in sync as the server's surface changes.